

LexSG: Accelerating Lexicographical Permutations through Factorial-Based Structure Generation

Yusheng HU^{1*}

^{1*}Independent Researcher, Shangdi East Road, Beijing, 100085, China.

Corresponding author(s). E-mail(s): dr.huyusheng@gmail.com;

Abstract

The full permutation generation algorithm is a core fundamental operator for combinatorial optimization and high-performance computing. Conventional Next Permutation schemes represented by the Narayana-Pandita algorithm suffer from limited computational efficiency due to frequent while-loop addressing. This paper proposes an acceleration architecture named LexSG (Lexicographical Structure Generation) based on the factoradic number system. By investigating the mapping relationship between lexicographical evolution and the factoradic number system, LexSG enables direct localization of swap positions and fundamentally eliminates the search overhead inherent in traditional algorithms. Experiments demonstrate that the proposed architecture can significantly boost the execution efficiency of full permutation generation.

Keywords: Permutation Generation; Fractal Isomorphism; Combinatorial Computing

MSC Classification: 68Q25 , 05A05

1 Introduction

Currently, the industry-standard Narayana-Pandita algorithm [1] [2] is typically implemented using Nextpermutation, which generates permutations in lexicographical order through local swaps and suffix reversals. While it is highly praised for its logical simplicity and low space complexity, its performance limitations become particularly prominent in high-performance computing scenarios.

Specifically, the algorithm relies on iterative backtracking and requires multiple scan passes to locate the next pivot, introducing substantial conditional branching. This logic imposes significant penalties on modern pipelined processors, where branch

mispredictions severely degrade throughput in large-scale or high-frequency iteration tasks.

Building on the successful application of efficient factoradic arrays to accelerate total permutation generation in Position Pure [4] and Ring-Cascade-Permutation-Algorithm [5], we further extend this factorial-based optimization to lexicographical order generation.

To address these bottlenecks, we propose LexSG (Lexicographical Structured Generation), a novel architecture for permutation generation that replaces traditional iterative searching with a static mapping approach. By deriving the mathematical correspondence between lexicographical order and the factorial number system, LexSG eliminates the need for loop-based addressing; the next permutation is located directly by updating the factorial base. This paper details the design principles of the LexSG architecture and demonstrates its superior performance, highlighting the latency reduction achieved by eliminating loop-based overheads.

2 Constructive Analysis and Preliminaries

2.1 Constructive Analysis of Permutation Generation

Permutation generation is essentially a recursive element selection process. At each step i , an element is selected from the available set of $n - i$ elements. Choosing an element at index j ($0 \leq j < n - i$) determines the specific subset of the remaining $(n - i - 1)!$ permutations. This constructive process inherently mirrors the factorial number system: selecting elements by index j uniquely corresponds to the digits of a factoradic representation. By induction, this confirms that the search space is partitioned into exactly $n!$ distinct states, each mapping uniquely to a factoradic coordinate.

2.2 Preliminaries: Mathematical Properties of the Factorial Number System

The permutation generation algorithm can be mathematically described via the Factorial Number System. This system provides a standardized method for mapping an integer $X \in [0, n! - 1]$ to a specific permutation. For any set of n elements, there exists a bijection between the permutation state and an array $C = [C_0, C_1, \dots, C_{n-1}]$, where each component C_i satisfies the following constraint:

$$0 \leq C_i < n - i$$

(Note: C_{n-1} is conventionally 0).

The factorial number system possesses the following fundamental mathematical properties:

- **Weighted Positionality:** Each digit C_i in the factorial number system carries a weight of $(n - 1 - i)!$. The representation of an integer X can be derived via successive

division:

$$X = \sum_{i=0}^{n-1} C_i \times (n-1-i)!$$

- **Dynamic Radix Constraints:** Unlike fixed-radix systems (e.g., binary or decimal), the factorial number system employs a dynamic base. When a component C_i reaches its upper bound $n-i-1$, any subsequent increment resets the digit to zero and triggers a carry to the next significant position.
- **Lexicographical Isomorphism:** As the counter X increments from 0 to $n!-1$, the corresponding permutation sequence evolves in strict lexicographical order. The changes in each digit C_i directly reflect the displacement within the permutation combination space.

To illustrate the mathematical correspondence established above, we present the mapping between the factorial number system and lexicographical permutations for $N = 4$ in Table 1. As demonstrated, the bijection between the factorial representation and the permutation sequence S_4 reveals a rigid mathematical structure. This structure enables our LexSG architecture to perform state transitions via simple arithmetic increments, fundamentally bypassing the search-based bottlenecks of traditional algorithms.

Table 1 Mapping from Factorial Number System to Lexicographical Permutations ($N = 4$)

Rank	Fact.	Perm.	Rank	Fact.	Perm.	Rank	Fact.	Perm.	Rank	Fact.	Perm.
0	0000	1234	6	1000	2134	12	2000	3124	18	3000	4123
1	0010	1243	7	1010	2143	13	2010	3142	19	3010	4132
2	0100	1324	8	1100	2314	14	2100	3214	20	3100	4213
3	0110	1342	9	1110	2341	15	2110	3241	21	3110	4231
4	0200	1423	10	1200	2413	16	2200	3412	22	3200	4312
5	0210	1432	11	1210	2431	17	2210	3421	23	3210	4321

3 Review of the *Next Permutation* Algorithmic Logic

The *Next Permutation* algorithm aims to identify the immediate lexicographical successor of a given sequence. It advances the current state to the next smallest lexicographical order through two core operations: *swap* and *reverse*.

3.1 Swap Logic

The algorithm begins with a right-to-left linear scan to identify the first index i such that $nums[i] < nums[i+1]$.

- **Pivot Identification:** The index i marks the critical point where the sequence transitions from a descending to an ascending order. To the right of this index,

the subarray $nums[i + 1 \dots n - 1]$ is in strictly descending order, representing the extreme state of this local permutation.

- **Search and Permutation:** The algorithm searches for the smallest element $nums[j]$ in the subarray $nums[i + 1 \dots n - 1]$ that is greater than $nums[i]$. By swapping $nums[i]$ and $nums[j]$, we ensure that the prefix remains unchanged while the permutation is elevated to the next higher lexicographical level.

3.2 Reverse Logic

After the swap, the subarray to the right of index i remains in descending order. To ensure that the resulting permutation is the *immediate* successor, this subarray must be reordered.

- **Sequence Normalization:** The subarray $nums[i + 1 \dots n - 1]$ is reversed to transform it from descending to ascending order.
- **Local Minimization:** This operation resets the suffix following index i to its lexicographically minimal state. By preserving the prefix while minimizing the suffix, the algorithm advances the sequence to its immediate successor.

3.3 Algorithm Review

As shown in Algorithm 1, the algorithm contains two while-loops. Although certain compiler optimizations can be applied, frequent branch mispredictions still degrade its runtime performance.

Algorithm 1 Search-based Next Permutation Algorithm (reference only)

Require: An array A of length n

Ensure: The lexicographically next permutation in A

```

1:  $i \leftarrow n - 2$ 
2: while  $i \geq 0$  and  $A[i] \geq A[i + 1]$  do
3:    $i \leftarrow i - 1$ 
4: end while
5: if  $i < 0$  then
6:   return false
7: end if
8:  $j \leftarrow n - 1$ 
9: while  $A[j] \leq A[i]$  do
10:   $j \leftarrow j - 1$ 
11: end while
12: swap( $A[i], A[j]$ )
13: reverse( $A[i + 1], A[n - 1]$ )
14: return true

```

4 Logical Correspondence of Swap and Reverse in LexSG

The evolution of lexicographical permutations exhibits a one-to-one correspondence with the components of the factorial number system. To elucidate these evolutionary patterns, we establish the following properties:

- **Property 1 (Ascending Sequence):** When all components of the factorial representation C are zero (i.e., $C = [0, 0, \dots, 0]$), the corresponding permutation is the original ascending sequence. *Logic:* In this state, each round of element selection consistently picks the minimum available value. Given the 0-based indexing, repeatedly selecting index 0 maintains the minimum value, resulting in a strictly ascending sequence.
- **Property 2 (Descending Sequence):** When C reaches its maximum state, $C_i = n - 1 - i$, the resulting permutation is strictly descending. *Logic:* Each selection extracts the maximum available element. This aligns with the factorial carry mechanism, where state migration in descending permutations is only triggered when a local maximum is reached.
- **Property 3 (State Migration):** Permutation state evolution is driven by incrementing the factorial components via two primary operations:
 - *Minimal Granularity Evolution:* Incrementing C_{n-2} from 0 to 1. Since C_{n-1} is always 0, this triggers the smallest possible lexicographical shift at the sequence tail.
 - *Carry Cascading:* When a component C_i reaches its upper bound $n - 1 - i$, it resets to zero and carries to the next significant position C_{i-1} . This corresponds to the critical point where a local maximum jumps to the next larger lexicographical state.

4.1 Factorial Evolution Operators: From Iteration to Mapping

By analyzing the increment behavior of C_i , we map traditional *while*-loop logic to deterministic algebraic operators:

- **Property 4 (Minimal Swap Operator):** Incrementing C_{n-2} from 0 to 1 involves only a swap of the last two elements. In the symmetric group space, this is a minimal track shift $\{\dots, a, b\} \rightarrow \{\dots, b, a\}$, equivalent to a minimal-scale *Reverse* on a suffix of length 2.
- **Property 5 (Cascading Mapping Operator):** When C_K overflows ($0 \leq K < n - 1$), the permutation undergoes a composite *Swap and Reset* transformation.
 - *Locating Operator:* A carry corresponds to jumping to the next state of C_K . We select the C_K -th element from the remaining candidate set $\{D_K, D_{K+1}, \dots, D_{n-1}\}$ and place it at position K .
 - *Single-step Mapping:* Since the subsequence $D_{K+1..n-1}$ is already sorted, a single swap with the element at D_K achieves the state jump.
 - *Reset Operator:* The suffix $D_{K+1..n-1}$ is forced to its lexicographical minimum (ascending order) via a direct *Reverse* operation.

4.2 Architectural Innovation: De-looping the Logic

Our derivation reveals a fundamental paradigm shift:

- **Search to Calculation:** LexSG replaces search-based pivot detection (i, j) with direct index calculation via factorial increments.
- **Operator Decoupling:** Complex branching is decomposed into discrete *Swap* and *Reverse* algebraic operators.
- **Deterministic Execution:** By eliminating global sequence scans in favor of a unique factorial base mapping, LexSG achieves a transition from $O(N)$ search complexity to constant-time $O(1)$ component calculation.

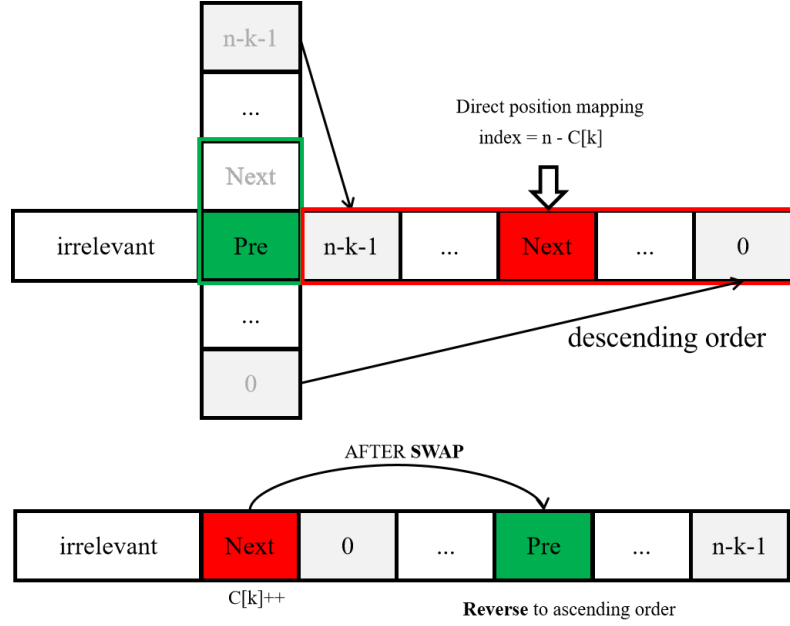


Fig. 1 LexSG Algorithm

As illustrated in Figure 1, instead of executing traditional linear scans, LexSG leverages the factoradic state $C[k]$ to achieve a direct position mapping via the function $\text{index} = n - C[k]$, instantly pinpointing the target successor (*Next*). Furthermore, since the predecessor (*Pre*) and the designated suffix partition are localized within the current state transition, executing the subsequent swap-and-reverse operations strictly updates the local lexicographical boundary without affecting the irrelevant prefix segments.

5 LexSG: Overall Architectural Design

5.1 Design Philosophy: Factorial-Based Initialization

The LexSG architecture aims to efficiently generate permutations using the factorial system. By establishing a strict mathematical correspondence between the incremental behavior of factorial bases and the evolution of lexicographical order, LexSG eliminates redundant search operations common in traditional algorithms.

5.2 Algorithm LexSG

Algorithm 2 LexSG: Lexicographical Permutation Generation

Require: n (permutation order)

Ensure: All permutations of set $\{0, 1, \dots, n-1\}$

```
1: Initialize  $arr \leftarrow \{0, 1, \dots, n-1\}$ ,  $C[0 \dots n-1] \leftarrow 0$ 
2: Output permutation  $arr$ 
3: while true do
4:    $C[n-2] \leftarrow 1$ 
5:    $\text{swap}(arr[n-2], arr[n-1])$ 
6:   Output permutation  $arr$ 
7:    $C[n-2] \leftarrow 0$ 
8:    $i \leftarrow n-3$ 
9:   while  $i \geq 0$  and  $C[i] \geq (n-1-i)$  do
10:     $C[i] \leftarrow 0$ 
11:     $i \leftarrow i-1$ 
12:   end while
13:   if  $i < 0$  then
14:     break
15:   end if
16:    $C[i] \leftarrow C[i] + 1$ 
17:    $\text{swap}(arr[i], arr[n-C[i]])$ 
18:    $\text{reverse}(arr[i+1], arr[n-1])$ 
19:   Output permutation  $arr$ 
20: end while
```

5.3 Architectural Feature: Element-Agnosticism

LexSG features complete element-agnosticism, strictly decoupling its control flow from the data layer. State transitions depend exclusively on the factorial number system, rendering the core algorithm indifferent to the actual element types or values.

- **Elimination of Value Comparisons:** Traditional methods scan array values to evaluate relational constraints (e.g., $nums[i] < nums[i+1]$). Conversely, LexSG determines swap positions solely via the abstract state of the component sequence C_i , bypassing data-level comparisons entirely.

- **Type-Invariant Execution:** Because the control logic is divorced from the payload, LexSG exhibits a uniform execution path across diverse domains. Permuting non-symmetric or complex objects requires only a one-time index mapping, without specialized comparators.
- **Data-Independent Branching:** This agnostic design removes all data-dependent conditional branches. Decoupling the data layout from the control flow eliminates branch mispredictions and optimizes low-level instruction pipelining.

5.4 Algorithmic Complexity Analysis

Although existing literature frequently cites the amortized time complexity of the *Next Permutation* algorithm as $O(1)$, its practical execution is hampered by non-negligible constant-factor overheads stemming from conditional branching and iterative scanning. For reference, the logic of the *Next Permutation* algorithm is summarized in Algorithm 1.

Our complexity evaluation of the LexSG algorithm is as follows:

- **Computational Complexity:** LexSG eliminates the traditional reliance on iterative loops to locate swap targets. By leveraging the factorial number system, it achieves direct determination of state transition points in $O(1)$ time per component update.
- **Irreducibility of Operations:** As established by the *Next Permutation* logic, the transition between states in lexicographical order is non-linear; specifically, jumping from a local maximum to the next lexicographical state necessitates a complete reset of the suffix. LexSG retains the essential *Reverse* operation because it is mathematically irreducible within the lexicographical paradigm. Consequently, LexSG and the traditional algorithm share the same asymptotic time complexity.
- **Constant Factor Optimization:** LexSG shifts the state transition paradigm from explicit “search-and-backtrack” loops to direct factoradic-based “mapping-and-assignment.” This structural change removes data-dependent branching, minimizing CPU pipeline stalls and reducing the algorithmic constant factor to achieve high execution throughput.

6 LexSG Performance Benchmarks

6.1 Experimental Setup

The performance evaluation was conducted on a machine equipped with an Intel Core i7-10750H processor (@ 2.60GHz, 6 cores/12 threads, 12MB L3 cache) and 64GB of DDR4-3200 dual-channel RAM, running Windows 11 (64-bit). To ensure consistency and eliminate architecture-dependent noise, all algorithms were compiled using Clang 22.1.5 with the optimization flags `-O3 -std=c++20`.

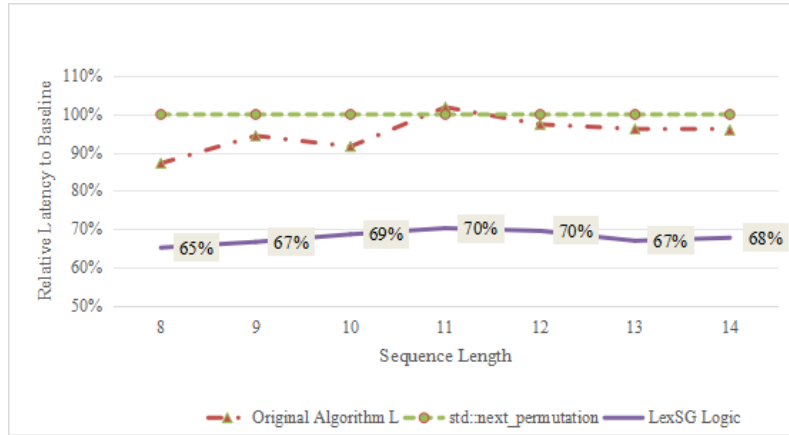
6.2 Performance Evaluation Data

Table 2 presents the execution time and latency (measured in nanoseconds per permutation) across sequence lengths ranging from $N = 8$ to $N = 14$.

Table 2 Performance comparison across different sequence lengths (N)

N	Algorithm	Time (s)	Speed (ns/perm)
8	<code>std::next_permutation</code>	0.0001385	3.43502
	Original Algorithm L	0.0001208	2.99603
	LexSG Logic	0.0000902	2.23710
9	<code>std::next_permutation</code>	0.0011801	3.25204
	Original Algorithm L	0.0011139	3.06961
	LexSG Logic	0.0007864	2.16711
10	<code>std::next_permutation</code>	0.0132084	3.63988
	Original Algorithm L	0.0120976	3.33377
	LexSG Logic	0.0090567	2.49578
11	<code>std::next_permutation</code>	0.1336680	3.34867
	Original Algorithm L	0.1362020	3.41214
	LexSG Logic	0.0938412	2.35092
12	<code>std::next_permutation</code>	1.6358800	3.41518
	Original Algorithm L	1.5932000	3.32609
	LexSG Logic	1.1369800	2.37364
13	<code>std::next_permutation</code>	21.519800	3.45588
	Original Algorithm L	20.708700	3.32561
	LexSG Logic	14.393900	2.31152
14	<code>std::next_permutation</code>	303.26800	3.47871
	Original Algorithm L	290.79100	3.33559
	LexSG Logic	205.42600	2.35639

6.3 Test Result Analysis

**Fig. 2** Total Execution Time from 8 to 14

As illustrated in Figure 2. Average latency per permutation normalized to the `std::next_permutation` benchmark. The y-axis represents the latency percentage relative to the baseline. LexSG maintains a stable performance advantage, requiring approximately 67–70% of the time required by the standard implementation.

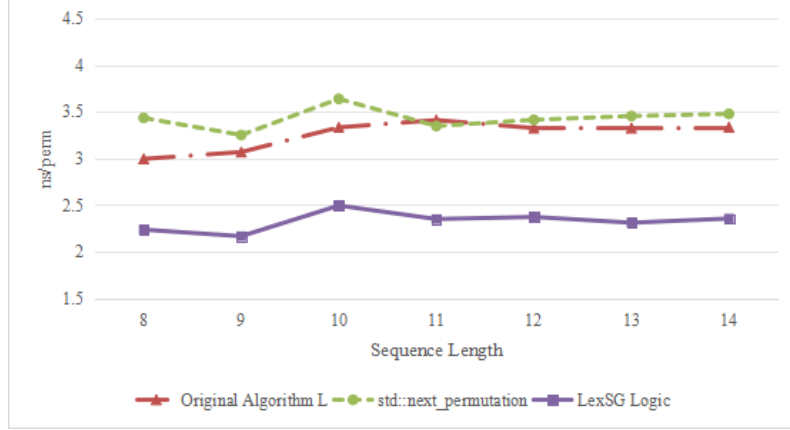


Fig. 3 Latency per Permutation from 8 to 14

As illustrated in Figure 3, the LexSG architecture maintains a stable latency of approximately 2.3 ns/permutation across the tested range of $N = 8$ to 14. This performance margin remains steady throughout our evaluation, confirming that the benefits of our mapping-based design are scalable and effectively sustain architectural efficiency across these problem sizes.

7 Conclusion and Future Work

This paper presented LexSG, a novel permutation generation architecture that models lexicographical evolution as a discrete mathematical mapping based on the factorial number system. By enabling direct functional computation of state transitions, LexSG eliminates the traditional reliance on global scanning and iterative *while*-loops. Performance analysis validates that this decoupling drastically reduces algorithmic constant overhead and data-dependent branching, offering an efficient primitive for high-performance combinatorial computing.

Future research will pursue two primary directions:

1. **Parallelization and Scaling:** While single-threaded lexicographical methods (e.g., *next_permutation*) are traditionally slower than Heap’s algorithm [3], LexSG can overcome this limitation through parallelization. By leveraging its deterministic state mapping for block-partitioned parallelization, we aim to extend LexSG to multi-core and cluster environments, achieving a thread-level performance leap that surpasses sequential alternatives.

2. **Multiset Permutations:** Adapting the architecture to sequences with duplicate elements, focusing on overcoming topological de-duplication bottlenecks while preserving the zero-branch, pipeline-efficient execution path.

References

- [1] Robert Sedgewick. Permutation generation methods. *Computing Surveys*, 9:137–164, 1977.
- [2] Donald E. Knuth. *The Art of Computer Programming, Volume 4, Fascicle 2: Generating All Tuples and Permutations*. Addison-Wesley Professional, Reading, MA, 2005.
- [3] B. R. Heap. Permutations by interchanges. *The Computer Journal*, 6(3):293–294, 1963.
- [4] Yusheng Hu. Position Method: A Linear-Time Generation Algorithm for Permutations. *SSRN*, February 22, 2026. Available at <https://ssrn.com/abstract=6285581> or <http://dx.doi.org/10.2139/ssrn.6285581>.
- [5] Yusheng Hu. Ring-Cascade: A Sub-constant Amortized Time Algorithm for High-Speed Permutation Generation. *SSRN*, April 09, 2026. Available at <https://ssrn.com/abstract=6553539> or <http://dx.doi.org/10.2139/ssrn.6553539>.